

Synthesizability classification of generated structures

uvsib

Contents

1 What it is	1
2 Where it sits	1
3 The three classifiers	2
3.1 1. thermo — thermodynamic accessibility	2
3.2 2. reaction — precursor → product selectivity	2
3.3 3. pu — positive-unlabeled synthesizability	2
3.4 Combined verdict	3
4 Output	3
5 Configuration (input.yaml)	3
6 Scope note	3
7 Finite-temperature extension (phonons → Gibbs hull)	3
8 Precursor / synthesis-route literature search (containerized agent)	4
8.1 Where it sits	4
8.2 The agent	4
8.3 I/O contract	5
8.4 Gating, output, configuration	5
9 Status	6

1 What it is

A MainWorkChain stage that classifies **every generated structure** of a composition by synthesizability, using three independent views (the three the literature splits into). It runs after the phase diagram is built and before the surface/adsorbate stages, so the 0 K hull it needs already exists.

The core classification is post-processing only — no remote jobs. It reads the generated structures and their MLIP energies from the database, builds the hull, scores each structure, and writes the scores back. Two **opt-in** extensions go further and do submit remote work: the finite-temperature (Gibbs) hull (phonons, below) and the **precursor / synthesis-route literature search** — a containerized web-search agent (below).

2 Where it sits

```
MainWorkChain
  pd_ml (generates + relaxes structures, builds hull)
  pd_verification
```

```
--> synthesizability <-- this stage (workchains/main.py)
sqz
surface_builder
adsorbates
nano_generator
```

Gated like the other stages: skipped for the sqz and nano paths, skipped if `step_status["synthesizability"] == "Done"`, and globally toggled by `settings.SYNTH_ENABLED (input.yaml: synthesizability.enabled)`.

3 The three classifiers

All three live in `workchains/_synth_classifiers.py` (pure pymatgen/numpy, unit tested); the AiiDA wrapper is `workchains/synthesizability.py`.

View	Question	Signal
thermo	Is it thermodynamically accessible?	energy above the 0 K hull vs a metastability window
reaction	Will it form rather than its competitors?	formation energy from elemental precursors + the decomposition products it would form instead + polymorph selectivity at its composition
pu	Does it look like things that get made?	a positive-unlabeled learned score (loaded model, or a transparent surrogate)

3.1 1. thermo — thermodynamic accessibility

`e_above_hull` from `PhaseDiagram.get_e_above_hull`. Labels: product (on the hull, $\leq e_{\text{hull_tol}}$), metastable (within `meta_window` above), or inaccessible. Score = $\exp(-e_{\text{above_hull}} / \text{thermo_scale})$.

The window matters: metals/alloys sit at the tight end of the Sun/Ceder metastability scale, so `meta_window` defaults to 0.05 eV/atom — and the same caveat the literature flags applies: polymorph gaps can be below both MLIP and DFT-functional noise, so near-hull calls are exactly where a DFT check is worth its cost.

3.2 2. reaction — precursor → product selectivity

`PhaseDiagram.get_form_energy_per_atom` (reaction energy from the elemental precursors) and `get_decomp_and_e_above_hull` (the competing phases the composition decomposes into — the “products” that form instead). Plus a **polymorph gap**: the energy distance to the best generated structure at the same composition (0 $\Rightarrow$ this is the preferred polymorph). Labels: selective_product / competes / outcompeted.

E.g. an off-hull Cu_2Au is reported as decomposing into $0.667 \text{ Cu}_3\text{Au} + 0.333 \text{ CuAu}$ — the actual products, not just a number.

3.3 3. pu — positive-unlabeled synthesizability

Featurizes the composition (fraction-weighted mean/std of element properties: Z, electronegativity, group, row, mass, radius, Mendeleev number, + `n_elements`; `matminer-free`). If a pickled model with `predict_proba` is configured (`pu_model_path`), its probability is used. Otherwise a **transparent**

surrogate — $\text{sigmoid}(2 - 18 \cdot e_{\text{above_hull}} - 0.4 \cdot (n_{\text{elements}} - 2))$, flagged model: surrogate($e_{\text{hull}} + \text{complexity}$) — so the stage runs end-to-end without a trained model. Drop a real PU classifier (SynthNN / CSPML / SynCoTrain-style, fit on the same featurizer over ICSD-positives) at `pu_model_path` to replace it.

3.4 Combined verdict

Weighted average of the three scores (weights, default equal), with a hard rule: an on-hull, selectively-preferred phase is synthesizable regardless. Labels by `synth_cut` / `maybe_cut`: synthesizable / maybe / unlikely.

4 Output

Per structure, the full record (thermo, reaction, pu, combined) is merged into that generated DB-StructureVersion's attributes JSONB under "synthesizability". A run-level summary (counts per label + the ranked synthesizable candidates) is stored on the DBComposition row's attributes and emitted as the workchain's synthesizability Dict output.

5 Configuration (input.yaml)

```
synthesizability:
  enabled: true
  ehull_tol: 0.005      # eV/atom; on-hull tolerance
  meta_window: 0.05     # eV/atom; metastable window above the hull
  thermo_scale: 0.04    # eV/atom; accessibility score decay
  complexity_penalty: 0.4 # PU surrogate penalty per element beyond binary
  synth_cut: 0.66       # combined -> "synthesizable"
  maybe_cut: 0.33       # combined -> "maybe"
  # weights: [1.0, 1.0, 1.0] # thermo, reaction, pu
  # pu_model_path: /path/to/pu.pkl # trained PU model; omit -> surrogate
```

6 Scope note

Per the project scoping decision, the target space is metallic systems (alloys/intermetallics), not oxides/carbides — which is also where the hull, the metastability window, and the MLIP energies feeding this stage are most trustworthy. The honest limit is that all three views are **0 K energetics**: true synthesis-pathway/selectivity needs finite-T free energies (entropy, temperature) the potential can supply via phonons/MD but this stage does not yet compute.

7 Finite-temperature extension (phonons → Gibbs hull)

By default the three views score on the **0 K** hull. When `synthesizability.finite_T.enabled` is set, the stage first computes phonon free energies and scores on the **Gibbs hull at the synthesis temperature** — so vibrational entropy can flip which competing phase is the product.

Sub-step (per the user's spec):

```
near-hull candidates + hull vertices
-> PhononWorkChain (codes/phonon + codes/files/phonon.py)
    tight relax -> phonopy finite displacements -> MLIP forces
    -> force constants -> harmonic  $F_{\text{vib}}(T)$  -> QHA over a few volumes ->  $G(T)$ 
-> per-atom free-energy correction  $G(T) - E_0$  at the target T
-> finite-T (Gibbs) convex hull
```

-> classify on the finite-T hull

Only structures within `ehull_window` of the 0 K hull (plus the hull vertices) get phonons — far-above-hull phases won't be rescued by entropy, and phonons are expensive. Corrections are mapped back by structure UUID; the finite-T hull is rebuilt from all entries shifted by their $F_{\text{vib}}(T)$.

Hard requirement — conservative model. Phonons are second derivatives of the PES, so forces must be the analytic gradient of one energy. Use **MACE** or **MatterSim**; direct-force heads (eqV2/UMA, ORB) give asymmetric force constants and spurious imaginary modes and must not be used (`finite_T.model`, default MACE). Tight pre-relaxation (`fmax 1e-3`) avoids spurious imaginary modes; QHA captures the dominant thermal-expansion/Gibbs term without full anharmonic (TDEP/SCPH) treatment for most metals.

`synthesizability:`

```
finite_T:
  enabled: false
  temperature: 1000.0      # K, the synthesis-temperature hull
  model: MACE              # conservative MLIP ONLY
  ehull_window: 0.10      # eV/atom; phonons only for near-hull structures
  displacement: 0.01      # Å; do a convergence check per family
  volume_scales: [0.97, 0.99, 1.0, 1.01, 1.03]
```

Two honest limits (literature, 2024–26): (1) **absolute** vibrational free energies from foundation MLIPs carry a PES-softening bias (~meV/atom) that fine-tuning roughly halves — trust the stage for *ranking* (where error largely cancels), DFT-spot-check before quoting absolute entropies; (2) for metals the **electronic** free energy (Fermi smearing, electron-phonon) is non-negligible at synthesis T and is **not** in $F_{\text{vib}}(T)$ — a future term, not captured here.

8 Precursor / synthesis-route literature search (containerized agent)

All of the above is *model energetics* — can it exist, is it accessible. When `synthesizability.precursor_search.enabled` is set, the stage adds one more opt-in sub-feature after classification that asks the complementary, *experimental* question: **has anyone actually made this, and how?** It launches a **containerized agent** that web-searches recent publications for *proven* synthesis routes of the target material and returns the **DOIs** and **synthesis paths** it finds, for downstream processing.

8.1 Where it sits

`SynthesizabilityWorkChain`

```
  classify                <- the three views (+ optional finite-T hull)
  --> should_search_precursors <- gate (opt-in; see below)
      run_precursor_search  <- submit the containerized agent (one CalcJob)
      inspect_precursor_search <- store the DOIs + synthesis routes
  results
```

It is modeled on the remote-SQS step (`run_sqs`): a single `PrecursorSearchCalculation` `CalcJob` is submitted ad hoc rather than a sub-workchain — the search is single-shot and **skips gracefully** (the rest of the classification still stands) if a given deployment has not registered the plugin entry point or the `precursor_search` code.

8.2 The agent

`codes/precursor_search/container/` holds the real image: a minimal `debian-slim` (`glibc` + `ca-certificates` + `jq` + the self-contained Claude Code native binary; no Node/Python/git, ~310 MB) that runs **rootless under udocker** on the compute node. The precursor-agent wrapper drives

Claude Code headless, restricted to the **WebSearch / WebFetch** tools (no filesystem access), and always writes a schema-valid `output.json` even if the model returns prose — so the parser never trips. The reference stub `codes/files/precursor_agent.py` documents the same contract for offline use.

Because WebSearch/WebFetch run **server-side at Anthropic**, the only egress the compute node needs is HTTPS to the Anthropic API — not the open web.

8.3 I/O contract

The CalcJob stages a `request.json` and retrieves an `output.json`:

```
precursor-agent --request=request.json --output=output.json
```

- **request.json** — the composition (`chemical_formula`, `reduced_formula`, `elements`), the top synthesizable/maybe candidates for context, and the search knobs (`max_results`, `since_year`, `include_preprints`, `methods`).
- **output.json** — a dict with a `results` **list** (the only field the parser requires). Each result carries a `doi`, `title`, `year`, `url`, and a `synthesis_routes` list; each route gives `method`, `precursors`, `steps`, `conditions` (temperature / time / atmosphere), `product`, a `confidence`, and a verbatim evidence passage. The agent **never fabricates DOIs** — nothing verifiable found $\Rightarrow$ `results: []`.

8.4 Gating, output, configuration

The gate `should_search_precursors` is off unless `precursor_search.enabled` is true and, by default, spends the (paid, networked) search only on compositions with at least one **synthesizable** candidate (`only_synthesizable: true`; set false to always search). `context_candidates` caps how many top polymorphs are handed to the agent as context.

Results land in two places, next to the synthesizability summary: the workchain output node `precursor_search` (Dict), and `DBComposition.attributes["precursor_search"]` — the full payload, for downstream processing.

```
# input.yaml – opt-in sub-feature of synthesizability
synthesizability:
  enabled: true
  precursor_search:
    enabled: false           # flip on to run the search
    only_synthesizable: true # search only comps with a synthesizable candidate
    context_candidates: 5    # top polymorphs passed to the agent as context
    max_results: 20
    since_year: 2015
    include_preprints: true
    # methods: [solid-state, sol-gel, hydrothermal, flux, cvd, precipitation]

# config.yaml – register the container as a (Containerized)Code
codes:
  precursor_search:
    code_string: precursor_agent@<computer>
    job_script: {nodes: 1, ntasks: 1, cpus: 4, time: 3600}
```

Secrets stay out of provenance. The agent reads `ANTHROPIC_API_KEY` (or `CLAUDE_CODE_OAUTH_TOKEN`) from the **Computer's** environment / prepend_text or baked into the image — *never* a CalcJob input, so no credential ever enters the AiiDA provenance graph. Deploy detail lives in `codes/precursor_search/README.md` (plugin + contract) and `codes/precursor_search/container/README.m` (image build / udocker import).

9 Status

- Classifiers (`_synth_classifiers.py`): implemented, unit-tested on a metallic Cu-Au hull (on-hull preferred polymorph $\rightarrow$ synthesizable; +25 meV/atom worse polymorph $\rightarrow$ maybe; +75 meV/atom $\rightarrow$ unlikely; off-hull Cu₂Au $\rightarrow$ correct decomposition products). Finite-T hull tested to flip a ranking (a higher-entropy CuAu polymorph overtakes the 0 K winner at 900 K).
- Workchain + DB write-back + MainWorkChain wiring + entry points: implemented.
- Phonon runner (`codes/files/phonon.py`) + `codes/phonon/` plugin: implemented; the runner end-to-end tested with EMT on fcc Cu (ZPE +0.033 eV/atom at 0 K $\rightarrow$ -0.51 eV/atom at 1200 K, thermal expansion, no imaginary modes).
- Precursor search (`codes/precursor_search/` CalcJob + parser + workchain wiring + entry points): implemented and offline-tested (parser accepts the empty/populated contract, rejects malformed; agent honors `--request/--output` and the inspect step counts DOIs/routes). The real container (`codes/precursor_search/container/`, Claude Code headless WebSearch/WebFetch) builds and passes its 13/13 offline plumbing test; the live end-to-end run needs ANTHROPIC_API_KEY on the compute node and is still untested.
- Open seams: trained PU model (`pu_model_path`); MLIP phonon fine-tuning for quantitative absolute free energies; an electronic free-energy term for metals; a live end-to-end precursor-search run + downstream consumption of the stored DOIs/synthesis routes.